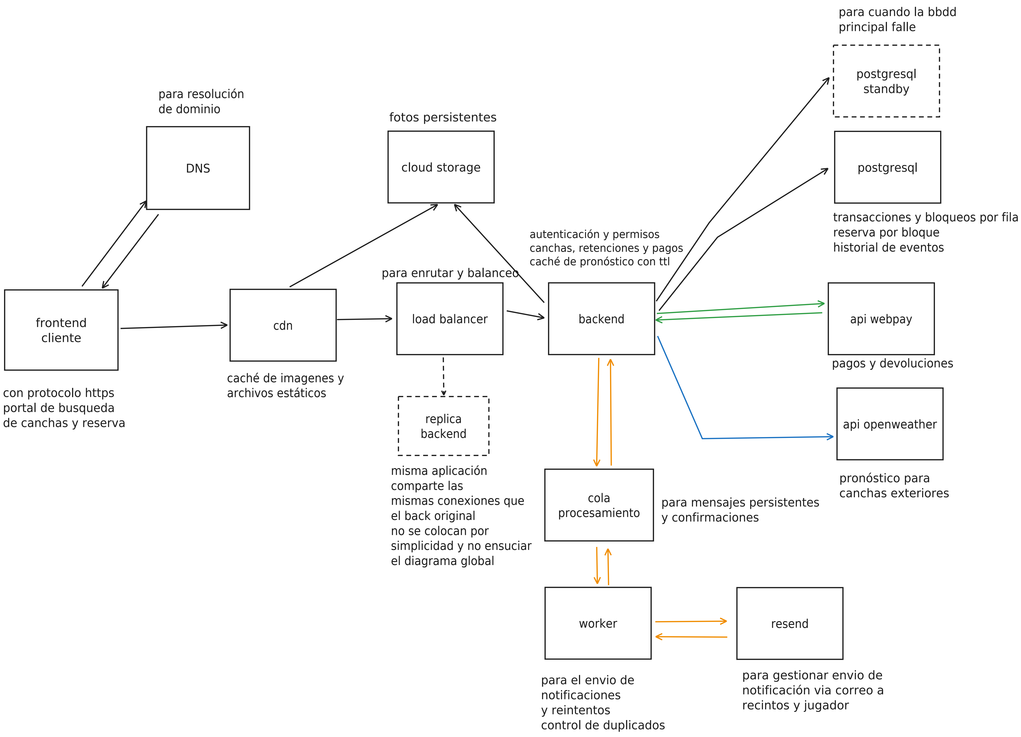

1.- supuestos base:
- Para los servicios usados consideraremos los servicios de infraestructura de aws con dos zonas de disponibilidad, us central y us west ya que se acepta una interrupción temporal ante una falla regional, el alcance geográfico del negocio sigue siendo la región metropolitana.
- El equipo conoce desarrollo web, fullstack y sql; además suponemos que el volumen inicial cabe en una instancia de postgresql, aún para mvp no se justifica distribuir los datos entre varias bbdd.
- El presupuesto permite dos instancias pequeñas de backend y postgreSQL standby; usar la nube no garantiza disponibilidad por sí solo.
- Correo electrónico será el canal de notificación. Se distingue el envío aceptado por el proveedor de la llegada efectiva al buzón que depende del servidor del destinatario.
- Las fechas serán almacenadas como instantes UTC y presentadas en America/Santiago; la ventana de "hoy y 7 días siguientes" se interpreta según fecha local.

1.1.- frontend: búsqueda y gestión de reservas

Proponemos una aplicación web, por ejemplo React, distribuida mediante S3 y CloudFront, que consume la API por HTTPS/JSON, permite buscar sin cuenta y exige autenticación para reservar. Consulta disponibilidad al abrir la ficha de la cancha del recinto y cambiar de fecha, dentro de los ocho días indicados, el backend la revalida al solicitar la retención, es la fuente de la verdad.

La desventaja es que un bloque mostrado como disponible puede ocuparse antes del clic: la interfaz debe comunicar el conflicto y actualizarse. WebSockets permitirían actualizaciones más inmediatas, pero añaden conexiones y sincronización, y tampoco eliminan la validación transaccional. Para el MVP, consultas bajo demanda cubren el requisito con menor mantenimiento.

1.2.- backend y réplicas

Un backend modular concentra autenticación, catálogo, reservas, pagos y coordinación de notificaciones. Dos instancias de la misma aplicación en ECS/Fargate comparten PostgreSQL, almacenamiento, cola y APIs externas; en el diagrama omití las conexiones de la réplica para facilitar la lectura.

Se reduce la coordinación técnica frente a microservicios y se aprovecha el conocimiento del equipo dentro de los seis meses. Las réplicas permiten continuar ante la caída de una instancia, a cambio de pagar capacidad adicional y mantener sesiones y operaciones críticas fuera de memoria local. Los microservicios permitirían despliegues independientes, pero multiplicarían contratos, monitoreo y fallas de comunicación. La autorización siempre se verifica en el servidor: cada recinto administra sus canchas y cada jugador accede a sus órdenes.

1.3.- load Balancer

Un load balancer entrega un punto de entrada y distribuye solicitudes entre las instancias del backend, teniendo health checks podemos detectar instancias caídas y dirigir tráfico hacia las restantes saludables (réplica) . Con esto aseguramos respaldar la operación y los despliegues, pero añade costo y configuración.

Una sola instancia sería más barata, pero dejaría la API indisponible durante su caída. Administrar Nginx también sería viable, aunque trasladaría al equipo su actualización y redundancia. El load balancer no reemplaza autenticación ni incorpora automáticamente rate limiting, los cuales implemetamos en el backend, siguiendo coherencia entre replicas.

1.4.- cdn y almacenamiento de imagenes

El recinto sube la imagen al backend, que comprueba permisos, formato y tamaño, la almacena en S3 y registra su referencia en PostgreSQL. Al consultar una cancha, el backend entrega la URL de CloudFront (el cdn de aws) ; el navegador solicita allí la imagen. CloudFront utiliza su caché o consulta S3 si necesita obtenerla, con eso evitamos tener que estarle pegando siempre al backend, evitando overhead de consultas por el mismo asset que no muy probablemente no ha cambiado.

1.5.- pronóstico y caché

El backend consulta OpenWeather únicamente para canchas exteriores y reutiliza el pronóstico por ubicación. Como propuesta inicial, guarda el resultado y su fecha de actualización en PostgreSQL con un TTL de 30 minutos, ajustable según uso y presupuesto (se eligió de forma arbitraria pero podría ser otro).

Esto limita llamadas cobradas y acelera la ficha, aceptando cierta antigüedad del pronóstico. Si OpenWeather falla, mostramos el último dato con su fecha o se informa que no está disponible, sin impedir reservar. Una caché dedicada, como ElastiCache, reduciría lecturas de PostgreSQL, pero incorporaría costo y operación adicionales. Para el volumen supuesto, reutilizar la base existente resulta más sencillo.

1.6.- concurrencia y retenciones

Proponemos bloqueo pesimista mediante SELECT ... FOR UPDATE, dentro de una transacción corta y con espera limitada. Se bloquean filas existentes del jugador y del bloque, siempre en el mismo orden; se valida que el bloque no haya comenzado, esté libre y el jugador no tenga otra retención vigente, se término registrando el vencimiento y se confirma la transacción.

Los bloqueos duran hasta finalizar la transacción; no permanecen durante los diez minutos de retención, para garantizar la reserva definitiva se incluye UNIQUE(bloque_id) como protección adicional.

Se evita que varios jugadores avancen al pago por el mismo bloque, reduciendo las comisiones adicionales descritas. La interfaz informa cuando el bloque ya está ocupado y permite elegir otro. El costo es la espera bajo alta competencia y el manejo de timeouts en spikes horarios como entre 19 -> 22hrs. El locking optimista sería viable, pero nos lleva a tener que resolver conflictos mediante actualizaciones condicionales y reintentos, SKIP LOCKED no conviene para adjudicar el bloque elegido. Como son reservas no podemos dejar que dos tengan el mismo bloque.

1.7.- cola, workers y notificaciones

Proponemos SQS + Lambda + Resend. PostgreSQL guarda la reserva y un evento pendiente en la misma transacción (outbox); un publicador envía ese evento a SQS, luego Lambda lo procesa y llama por HTTPS a Resend para notificar a jugador y recinto, así una caída entre guardar y publicar deja trabajo recuperable.

La integración administrada de Lambda con SQS maneja recepción y eliminación tras un procesamiento exitoso, pero admite procesamiento repetido, es importante aquí que el código sea idempotente.

Con Resend evitamos administrar servidores de correo y workers permanentes, a cambio de dependencia externa, costo por uso y manejo de reintentos. Resend se justifica por su API; SES sería una alternativa integrada en AWS, cuya configuración debe compararse según experiencia del equipo. Para cumplir el minuto, se limitaría el tiempo de espera de publicación y procesamiento, y se alerta por antigüedad de mensajes.

1.8.- postgreSQL y disponibilidad

Elegimos PostgreSQL por sus transacciones, restricciones y relaciones entre jugadores, canchas y reservas, además del conocimiento SQL supuesto. Un esquema inicial contemplaría modelos de:
- usuarios, recintos y miembros para gestionar identidad y permisos-
- canchas, fotos y horarios, para armar el catálogo
- bloques y retenciones para ver disponibilidad y garantizar exclusividad.
- reservas, pagos y devoluciones para gestionar las ordenes de reserva y trazabilidad monetaria.
- eventos pendientes y notificaciones para tener una vía de comunicación confiable y seguimiento de enviós.
- pronosticos, para reutilizar de forma temporal las consultas meteorológicas.

Se preservan monto y datos históricos necesarios; bajar una cancha no elimina reservas ni invalida retenciones existentes, así también los cambios de horario, no borran bloques comprometidos.

Para disponibilidad se propone una bbdd con una de reserva en standby, que permite failover y no sirve consultas de lectura en esa modalidad. Aumenta el costo, si, pero reduce el trabajo de recuperación. Se elige postgreSQL simplemente porque es el estandar mas usado con gran documentación, mantención y demás.
Importante considerar un connection pool para gestionar la bbdd y no colapsarla.

2.- Para responder a la pregunta 2, consideraría que para el crecimiento futuro emplearía un almacenamiento por niveles según la frecuencia de consulta y rangos de tiempo de la información. Los órdenes de los últimos 12 meses permanecerían en postreSQL usando un índice compuesto por el jugador y fecha para facilitar las consultas de cada uno, las anteriores a ese periodo se archivarían en S3, agrupadas igual por jugador y período, conservando en el mismo postgreSQL principal un catálogo pequeño con su ubicación, asi tenerlo mapeado. El back consultaría cada nivel según las fechas solicitadas. Así se consulta el período solicitado sin recorrer todo el historial.

Esta separación permite reducir el crecimiento de las tablas e índices operativos y permite conservar indefinidamente los registros antiguos sin mantener todo el historial en el postgreSQL. Como contrapartida, exige desarrollar y mantener el almacenamiento en S3, gestionar 2 rutas de consultas y pagar solicitudes y mantencion en el mismo S3. Eligiría una clase de acceso inmediato tipo Standard-IA, evaluando sus cargos y tamaños mínimos, de esta forma hacer algo tipo warm-cold data, warm para lo necesario 1 año y cold para lo que sea mas allá.

Para preservar la auditoria, trasladar la data de postgreSQL a S3 copiaría y verificaría los registros antes de retirar su copia operativa, asi conservamos los datos originales de la reserva aunque cambién o se elimine la cancha del catalogo por ejemplo. Esto a realizar en plazos según se corresponda como por ejemplo 1 vez a la semana, por mes o según las capacidades de presupuesto lo permitan, para sincronizar y trasladar de postgreSQL a S3.

Mantener todo en PostgreSQL sigue siendo la mejor alternativa al comienzo pero si se desea escalar, es necesario tener 2 mecánismos de selección de datos para la data fresca y la mas antigua como se comentó anteriormente, cuesta mas en términos de infraestructura y mantenibilidad pero es lo mas robusto en el largo plazo.

3.- Declaración del uso de IA.
El diagrama se realizó derechamente a mano, sin guía de IA, se utilizó IA para iterar y reflexionar los tradeoffs y argumentos sobre los elementos de la arquitectura pero las redacciones fueron realizadas a mano luego de la debida lectura y consolidación de la información.

4.- Referencias/Bibliografia
1.- se utilizó este articulo como referencia https://medium.com/@abhirbkulkarni/designing-scalable-booking-system-with-microservice-architecture-26a4a5b7337e
2.- la ppt de clases 02- UC - System Design 2026 - Clase 1 - 5
3.- algunos conceptos del libro system design the big archive - byte byte go
4.- algunas ideas del libro Designing data-intensive applications - Kleppmann
5.- Apuntes propios tomados de las clases.